ABAP/UI5 Review Assistant

Vollständiges Struktur-, Projekt- und Ausbaudokument analog zum Field Change Accelerator

Zielbild: interner Accelerator für SAP ABAP, RAP, CDS, OData, SAP Fiori Elements und SAPUI5 zur beschleunigten, strukturierten und konsistenten Review-Arbeit über Snippets, Diffs, Lösungsentwürfe, Änderungspakete und kleine bis mittlere Fullstack-Anpassungen hinweg.

Dokumentcharakter: Dieses Dokument ist bewusst nicht verdichtet. Es enthält die vollständige Struktur, alle diskutierten und naheliegenden Lösungsansätze, sämtliche sinnvollen Ausbaustufen, Projektphasen, Funktionsblöcke, Risiken, Architekturvarianten und Delivery-Empfehlungen für einen ABAP/UI5 Review Assistant.

Inhaltsübersicht

- 1. Produktdefinition und Zielbild
- 2. Einordnung im Accelerator-Portfolio
- 3. Problemraum und typische Review-Anlässe
- 4. Vollständiger fachlicher und technischer Umfang
- 5. Kernfunktionen im Detail
- 6. Lösungsansätze und Architekturvarianten
- 7. Tech-Stack-Optionen und Empfehlung
- 8. UI-Ansatz, Informationsarchitektur und Komplexität
- 9. Datenmodell, Input- und Output-Schemata
- 10. Rule Packs, Pattern Libraries und Wissensbausteine
- 11. Ausbaustufen: MVP, v1, v2, Premium
- 12. Feature-Backlog mit Priorisierung
- 13. Roadmap, Delivery-Plan und Projektorganisation
- 14. Qualitätskriterien, Risiken und Anti-Goals
- 15. Beispiel-Fälle und erwartete Ergebnisse
- 16. Erfolgsmessung und nächste Ausbauschritte

1. Produktdefinition und Zielbild

Der ABAP/UI5 Review Assistant ist kein autonomer Entwickler und kein generalistischer Code-Reviewer für beliebige Sprachen. Er ist ein spezialisierter interner Accelerator für SAP-nahe Entwicklungsreviews im Spannungsfeld von ABAP, RAP, CDS, OData, Fiori Elements und SAPUI5. Sein Kernnutzen liegt darin, Review-Arbeit zu standardisieren, blinde Flecken zu reduzieren und aus verstreuten Einzelbeobachtungen eine belastbare technische Review-Ausgabe zu erzeugen.

Kernidee: Ein Review Assistant nimmt ein Snippet, einen Diff, eine Lösungsskizze, eine technische Anfrage oder ein kleines Änderungspaket entgegen und liefert daraus priorisierte Findings, Risiken, Rückfragen, Testlücken, Refactoring-Hinweise, Clean-Core-nahe Warnungen sowie eine umsetzbare nächste Schrittfolge.

- Read-only Unterstützer statt schreibender Autopilot

- Fokus auf Review-Qualität, Vollständigkeit und Priorisierung statt auf autonome Korrekturen
- Geeignet für Voranalyse, Peer Review, Pre-Merge-Check, Delivery-Review und Ticket-to-Solution-Nachschärfung
- Besonders wertvoll bei wiederkehrenden Mustern in ABAP/RAP/Fiori/UI5-Änderungen

Das Produkt soll nicht Entwicklerurteil ersetzen. Es soll dafür sorgen, dass technische Reviews schneller, reproduzierbarer und qualitativ konsistenter werden – besonders dann, wenn kleine bis mittlere Änderungen unter Zeitdruck umgesetzt werden.

2. Einordnung im Accelerator-Portfolio

Im Gesamtbild der besprochenen Accelerators besetzt der ABAP/UI5 Review Assistant die Qualitäts- und Kontrollschicht. Während der Ticket-to-Solution Assistent die Analysephase beschleunigt, der Field Change Accelerator Änderungen strukturiert und der RAP/Fiori Skeleton Generator das leere Blatt beseitigt, sorgt der Review Assistant dafür, dass vorgeschlagene oder bereits begonnene Änderungen sauber, verständlich, testbar und wartbar bleiben.

- Vor dem Coding: Review von Lösungsansätzen, Architekturannahmen und Änderungsskizzen
- Während des Codings: Review von Snippets, Artefaktentwürfen, RAP-Design, Annotationen und UI5-Fragmenten
- Nach dem Coding: Pre-Merge-Review, Qualitätscheck, Testlückenanalyse, Delivery-Review
- Als Querschnittswerkzeug: gemeinsame Qualitätssprache für ABAP-, RAP- und UI5-Arbeit

Gerade in deinem Profilkontext – ABAP, SAPUI5/Fiori, RAP/CDS/OData, EWM/Yard/Service sowie integrationsnahe SAP-Projekte – entsteht der Hebel nicht primär durch autonome Generierung, sondern durch konsequent bessere Review-Disziplin.

3. Problemraum und typische Review-Anlässe

Der Assistant adressiert keine beliebigen Software-Reviews, sondern ein enges, SAP-spezifisches Muster. Typisch sind kleine bis mittlere Änderungspakete, heterogene Technologie-Mischungen und knappe Zeitfenster, in denen formale Codequalität, fachliche Konsistenz und Testabdeckung leicht auseinanderlaufen.

3.1 Typische Review-Anlässe

- ABAP-Snippet oder Diff prüfen
- UI5-Controller, View oder Fragment auf Struktur- und Wartbarkeitsprobleme prüfen
- RAP-Artefakte auf Konsistenz zwischen CDS, Behavior, Service und UI hin überprüfen
- Fiori Elements Erweiterung auf Annotationen, Filter/Search-Verhalten und Seiteneffekte prüfen
- Lösungsentwurf oder Change-Plan gegen wahrscheinlich vergessene Schichten gegenlesen
- Pre-Merge-Review für kleines Änderungsset
- Bugfix-Review, bei dem Nebeneffekte, Tests und Lesbarkeit kritisch sind
- Clean-Core-nahe Vorprüfung: Nutzung released APIs, Side Effects, technische Schulden

3.2 Warum diese Reviews anspruchsvoller sind als sie wirken

- ABAP-/RAP-/UI5-Mischszenarien verteilen Probleme über mehrere Schichten
- Viele Fehler sind keine Syntaxprobleme, sondern Struktur-, Kopplungs- oder Wartbarkeitsprobleme

- Review-Relevanz hängt stark vom Kontext ab: Snippet, Diff, App-Typ, Prozess, Rollenkonzept, bestehende Architektur
- Unter Zeitdruck werden Tests, Fehlerszenarien, Rollenlogik und Annotationen oft unvollständig betrachtet
- Ein guter SAP-Review erfordert andere Heuristiken als ein generischer Linter

4. Vollständiger fachlicher und technischer Umfang

4.1 In Scope

- Read-only Review von ABAP, RAP, CDS, OData, Fiori Elements und SAPUI5-Artefakten
- Review von Snippets, Diffs, Lösungsentwürfen, Tickets und kleinen bis mittleren Änderungspaketen
- Findings zu Struktur, Wartbarkeit, Lesbarkeit, Testbarkeit, offensichtlichen Risiken und typischen SAP-Smells
- Testlückenanalyse und Vorschlag naheliegender Testfälle
- Refactoring-Hinweise mit realistischer Priorisierung
- Clean-Core-nahe Hinweise und released-API-/Entkopplungs-Sensibilisierung
- Rückfragen bei unklarem fachlichen oder technischen Kontext
- Umsetzungs- bzw. Review-Checklisten für nächste Schritte

4.2 Out of Scope im Grundsystem

- Autonome, unüberwachte Codeänderungen
- Vortäuschen echter Repository-Kenntnis ohne Code- oder Systemzugriff
- Verbindliche Performance-Aussagen ohne belastbaren Kontext oder Messung
- Security-Freigaben, Compliance-Freigaben oder Berechtigungsabnahmen
- Vollständige Architekturabnahmen für große Programme oder ganze SAP-Landschaften
- Automatische Transportplanung oder Systemdeployment
- Generische Code-Modernisierung ohne klare Review-Fragestellung

4.3 Unterstützte technische Kontexte

- ABAP-Backend allgemein, inkl. ABAP OO, klassische Reports, Klassen und Utility-Logik
- CDS Views, View Entities, Projection / Consumption / Metadata Extensions
- RAP: Behavior Definition, Actions, Determinations, Validations, Draft-nahe Fragestellungen
- Service Definition, Service Binding und OData-Exposition
- Fiori Elements: List Report, Object Page, Filterbar, Search, Actions, Annotationen
- SAPUI5 Freestyle: XML Views, Controller, Fragments, Binding, Formatter, Event-Handler-Struktur
- Mischszenarien, in denen deklarative Fiori-Ansätze und Freestyle-Code nebeneinander stehen

5. Kernfunktionen im Detail

5.1 Intake- und Review-Typ-Erkennung

Der Assistant muss zuerst erkennen, was überhaupt geprüft werden soll. Ohne diese Typisierung bleibt jede Review-Ausgabe zu generisch. Der Review-Typ steuert Schwerpunkte, Heuristiken, Schweregrade und notwendige Rückfragen.

- Snippet Review
- Diff Review

- Pre-Merge Review
- Lösungsentwurf Review
- Ticket-/Anfrage-basierter Vorreview
- Regression-Risk Review
- Clean-Core-/Architecture-Sanity-Check

5.2 Artefakt- und Kontextklassifikation

Neben dem Review-Typ muss das System die Artefaktfamilie erkennen. Ein UI5-Controller braucht andere Regeln als ein CDS-View-Entwurf oder ein Behavior-Fragment. Ebenso wichtig ist die Unterscheidung zwischen Greenfield, Erweiterung, Bugfix und Refactoring.

- ABAP Klasse / Methode / Report
- CDS / Projection / Annotation
- Behavior Definition / Implementation
- Service / OData
- UI5 View / Controller / Fragment / Formatter
- gemischtes Fullstack-Änderungspaket

5.3 Findings Engine

Kern des Produkts ist die Findings Engine. Sie soll nicht nur Fehler auflisten, sondern verwertbare, priorisierte Review-Befunde erzeugen. Ein Finding braucht mindestens Schweregrad, Beobachtung, Begründung, mögliche Folge, Rückfrage oder Verbesserungsidee.

- kritisch: funktionale oder erhebliche Wartbarkeitsrisiken
- wichtig: starke Verbesserungsempfehlung
- optional verbessern: Lesbarkeit, Konsistenz, saubere Struktur
- unklar: Rückfrage statt scheinbarer Gewissheit

5.4 Test-Gap-Analyse

Viele technische Reviews bleiben zu codezentriert und lassen Tests außen vor. Der Assistant soll deshalb zu jedem relevanten Befund prüfen, ob Tests fehlen oder offensichtlich zu schmal gedacht sind.

- ABAP Unit / service-nahe Tests
- Action- und Validation-Tests
- UI-Verhalten / Rollenverhalten / Filter / Search
- Regression durch Seiteneffekte oder Randfälle

5.5 Refactoring- und Verbesserungsvorschläge

Das Produkt soll keine Komplettumbauten fordern, sondern realistische nächste Schritte. Ziel ist nicht maximaler Perfektionismus, sondern Review-Hilfe, die unter Projektbedingungen umsetzbar bleibt.

- kleiner Fix
- gezielte Strukturverbesserung
- mittelfristiger Refactoring-Hinweis
- explizit kein sofortiger Umbau trotz möglicher Ideallösung

5.6 Risk- und Impact-Sicht

Ein gutes Review benennt nicht nur Codequalitätsmängel, sondern auch Liefer- und Betriebsrisiken. Beispielsweise: unklare Seiteneffekte, fehlende Fehlerbehandlung, ungedeckte Rollenlogik oder versteckte Kopplung zwischen Service und UI.

- Risiko auf Funktion
- Risiko auf Wartbarkeit
- Risiko auf Testbarkeit
- Risiko auf Upgrade-/Clean-Core-Fähigkeit

5.7 Review-Zusammenfassung und nächste Schritte

Die Ausgabe muss am Ende in einen klaren Handlungsvorschlag münden: Was ist sofort zu klären, was jetzt zu ändern, was kann später verbessert werden und welche Tests fehlen noch?

- Top Findings
- offene Rückfragen
- empfohlene Korrekturreihenfolge
- Testergänzungen
- Go/No-Go bzw. 'Go with caveats' Einschätzung für kleine Changes

6. Lösungsansätze und Architekturvarianten

Für den ABAP/UI5 Review Assistant gibt es mehrere plausible Lösungsansätze. Nicht jeder Ansatz passt gleich gut zu deinem Ziel, interne Accelerators für Delivery-Geschwindigkeit zu bauen.

6.1 Prompt-only Agent

Der schnellste Start ist ein reiner KI-Agent mit klaren Rolleninstruktionen und festen Ausgabeformaten. Vorteil: sehr schneller Start und hohe Flexibilität. Nachteil: geringere Konsistenz, schwächere Nachvollziehbarkeit, stärkere Abhängigkeit von Prompt-Qualität.

6.2 Regelbasierter Reviewer

Ein rein regelbasierter Reviewer nutzt Pattern- und Rule Packs ohne generatives Modell. Vorteil: hohe Stabilität und gute Erklärbarkeit. Nachteil: schwache Generalisierung und hoher Pflegeaufwand bei abwechslungsreichen Inputs.

6.3 Hybrider Review Assistant

Der aus heutiger Sicht beste Ansatz ist ein hybrides System: strukturierte Intake-Schemas, feste Rule Packs, evidenzbasierte Heuristiken und ein LLM für Klassifikation, Verdichtung, Rückfragen und Formulierung der Findings. Dadurch bleiben Output-Struktur und Qualitätslogik berechenbar, ohne die Flexibilität zu verlieren.

6.4 Repo-aware Review Assistant

Eine spätere Ausbaustufe kann Repository- oder Diff-Kontext einlesen. Dadurch werden Findings präziser, weil reale Artefakte, Dateitypen und Änderungsgrenzen bekannt sind. Dieser Ansatz ist wertvoll, aber für den Start nicht zwingend.

6.5 CI-/Pipeline-integrierter Review Assistant

Langfristig kann der Assistant als Vorstufe zu Pull-Request-, Merge- oder Delivery-Prozessen laufen. Dann wird er vom ad-hoc Tool zum standardisierten Quality Gate.

Variante	Stärken	Schwächen	Empfehlung
Prompt-only Agent	schnellster Start, geringe Build-Kosten	uneinheitliche Qualität, wenig Robustheit	nur für frühe Exploration
Regelbasierter Reviewer	hohe Stabilität, gute Erklärbarkeit	zu starr für gemischte Inputs	nur als Teilbaustein sinnvoll
Hybrider Review Assistant	beste Balance aus Flexibilität und Konsistenz	etwas höherer Initialaufwand	klare Empfehlung für MVP/v1
Repo-aware Review	präzisere Findings, weniger Spekulation	braucht Integrationen und mehr Kontext	ab v2 interessant
CI-/Pipeline-Integration	skaliert Review-Prozesse im Team	operativ komplexer	späterer Ausbau

7. Tech-Stack-Optionen und Empfehlung

Der Review Assistant ist kein SAP-Produkt im engeren Sinne, sondern ein internes Analyse- und Qualitätssystem. Daher sollte der Tech Stack auf strukturierte Inputs, reproduzierbare Outputs und leichte Erweiterbarkeit optimiert werden.

7.1 Sinnvolle Kernoptionen

- Python + FastAPI für API, Orchestrierung und strukturierte Review-Pipelines
- Pydantic oder vergleichbare Schema-Modelle für Requests, Findings und Output-Kontrakte
- PostgreSQL mit JSONB für Beispiel-Cases, Rule Packs, Review-Historie und Feedback
- Markdown/YAML/JSON-basierte Rule Packs und Pattern Libraries
- LLM mit schemaerzwungenen Antworten für Klassifikation und Finding-Formulierung
- Optional später: Repo-/Diff-Parsing, semantische Fallähnlichkeit, CI-Integration

7.2 Alternative Stacks

- Node.js/TypeScript, wenn UI und API enger zusammenwachsen sollen
- Agent-first Setups in Claude Code, Codex oder OpenClaw für schnelle Exploration
- Nur CLI + Markdown + Prompts für die früheste Stufe ohne echte Anwendung

7.3 Empfehlung

Für ein belastbares internes Produkt ist ein hybrider Python/FastAPI-Ansatz mit strukturierter Domänenmodellierung und LLM-gestützter Findings-Generierung am sinnvollsten. Ein reines Agentensetup eignet sich gut zur Exploration, aber weniger als dauerhaftes Qualitätssystem mit History, Rule Packs und wiederverwendbarer Konsistenz.

8. UI-Ansatz, Informationsarchitektur und Komplexität

Die UI sollte keine generische Chat-Oberfläche sein. Für den Review Assistant ist eine technische Workbench sinnvoller: links Input und Kontext, rechts strukturierte Review-Ausgabe mit Filtern, Schweregraden, Testlücken und nächsten Schritten.

8.1 MVP-UI

- Single-page Review Workbench
- Input-Felder für Review-Typ, Artefakt-Typ, Kontext und Code/Snippet/Diff
- strukturiertes Ergebnis mit Blöcken für Findings, Tests, Risiken, Rückfragen und nächste Schritte
- Copy/Export nach Markdown

8.2 v1-UI

- History früherer Reviews
- Filter nach Kritikalität, Artefakt-Typ und Themencluster
- Variantenansicht intern technisch vs. management-/kundenfreundlich
- Review-Modus für vorhandene Lösungsentwürfe

8.3 v2/Premium-UI

- Diff-Upload und mehrteilige Artefaktauswahl
- Vergleich ähnlicher Alt-Fälle
- teamorientierte Notizen, Status und Review-Freigaben
- optionale Integration in PR-/Merge-Workflows

Die UI-Komplexität sollte niedrig starten. Der eigentliche Produktkern liegt in Findings-Qualität, Priorisierung und Struktur, nicht in aufwändiger Frontend-Orchestrierung.

9. Datenmodell, Input- und Output-Schemata

9.1 Zentrale Input-Arten

- Code Snippet
- Diff oder Patch-Auszug
- Lösungsbeschreibung / technischer Entwurf
- Ticket oder Kundenanfrage mit technischem Kontext
- Review eines bereits bestehenden Review-Ergebnisses

9.2 Sinnvolle Input-Felder

- review_type
- artifact_type
- context_summary
- code_or_diff
- goal_of_review
- technology_context
- known_constraints
- domain_pack
- question_focus wie performance, maintainability, clean-core, tests

9.3 Zentrale Output-Felder

- review_summary
- findings[]
- severity
- evidence_or_reasoning_basis
- missing_information[]
- test_gaps[]
- recommended_actions[]
- risk_notes[]
- clean_core_hints[]
- overall_assessment

9.4 Beispielhafte Schema-Objekte

Objekt	Zweck	Typische Felder
ReviewRequest	Eingabestruktur	review_type, artifact_type, context, content, constraints
Finding	einzelner Befund	severity, title, explanation, impact, recommendation
TestGap	fehlender Testfall	area, reason, suggested_test
RiskNote	Risiko-Hinweis	risk_type, consequence, mitigation
ReviewResult	gesamte Ausgabe	summary, findings, questions, actions, overall_assessment

10. Rule Packs, Pattern Libraries und Wissensbausteine

Der Assistant braucht keine allwissende Wissensbasis, sondern gut geschnittene, versionierbare Wissensbausteine.

10.1 Kern-Rule-Packs

- ABAP Readability und Structure Pack
- ABAP Error Handling und Testability Pack
- CDS / Annotation Review Pack
- RAP Consistency Pack
- Service / OData Exposure Pack
- Fiori Elements Review Pack
- UI5 Freestyle Review Pack
- Clean-Core / Released-API Awareness Pack

10.2 Pattern Library

- häufige ABAP-Smells: zu tiefe Verschachtelung, unklare Seiteneffekte, schlechte Benennung, unklare Exceptions
- häufige RAP-Smells: inkonsistente Behavior-Logik, fehlende Seiteneffekte, schwache Validierungen

- häufige CDS-/Annotation-Smells: unvollständige Exposition, unscharfe Search-/Filter-Nutzung, uneinheitliche Annotationen
- häufige UI5-Smells: Controller-Überladung, Logik in Views, fragmentiertes State-Management, fehlende Trennung von Verantwortlichkeiten
- häufige Fullstack-Smells: UI-/Service-Kopplung, vergessene Rollenlogik, fehlende Regressionstests

10.3 Domain Packs

- EWM / WM
- Yard Logistics
- Service
- MII/MES-nahe UI-Szenarien
- optional weitere Domänen wie Finance, PM oder MDM je nach Projektfokus

Domain Packs machen die Ausgaben realistischer. Sie ersetzen keine Systemkenntnis, helfen aber, typische Prozessfallen und erwartbare Artefaktlandschaften präziser zu adressieren.

11. Ausbaustufen: MVP, v1, v2, Premium

11.1 MVP

- Review-Typ erkennen
- Artefaktfamilie klassifizieren
- priorisierte Findings erzeugen
- Test-Gaps ableiten
- Rückfragen bei fehlendem Kontext stellen
- nächste Schritte und grobe Risiko-Sicht liefern
- Markdown-/JSON-artige strukturierte Ausgabe

11.2 v1

- stärkere Differenzierung zwischen ABAP, RAP, CDS, Fiori Elements und UI5 Freestyle
- größere Rule Packs und bessere Schweregradlogik
- Review von Lösungsentwürfen zusätzlich zu Code/Diffs
- History und wiederverwendbare Review-Templates
- bessere domänenspezifische Heuristiken

11.3 v2

- Repo- oder Diff-aware Analyse
- Cross-Artefakt-Review über mehrere Dateien hinweg
- Clean-Core-Sensibilisierung auf höherem Niveau
- Ähnlichkeitsvergleich mit früheren Fällen
- teamorientierte Review-Workflows und Status

11.4 Premium

- CI-/Merge-Integration
- optionale halbautomatische Fix-Vorschläge mit Human Review
- Review-Metriken und Lernschleifen
- Quality-Gates für wiederkehrende Change-Typen
- erweiterte Architektur- und Delivery-Review-Schicht

Stufe	Ziel	Nutzen	Nicht Ziel
MVP	brauchbare Review-Erstantwort	schnellere, konsistentere Reviews	keine autonome Korrektur
v1	höhere Präzision und Alltagstauglichkeit	weniger vergessene Aspekte, bessere Priorisierung	noch kein voll integriertes Quality Gate
v2	mehr Kontext und Teamfähigkeit	präzisere Multi-Artefakt-Reviews	kein Ersatz für erfahrenes Review im Grenzfall
Premium	prozessfähiges Quality System	skalierbare Review-Standards	keine vollautomatische Entwicklungsabnahme

12. Feature-Backlog mit Priorisierung

12.1 P0 / MVP

1. Review-Type Detection
2. Artifact-Type Classification
3. Structured Finding Generator
4. Severity Model
5. Question Engine
6. Test Gap Generator
7. Recommended Next Steps
8. Example Case Library

12.2 P1 / v1

9. RAP/CDS/UI5/Fiori-spezifische Rule Packs
10. Review von Lösungsentwürfen und Change-Plänen
11. Domain Packs
12. Template-Ausgaben intern vs. kundenfreundlich
13. History und Feedback Loop
14. bessere heuristische Komplexitäts- und Risikomodelle

12.3 P2 / v2

15. Diff- und Multi-Artefakt-Review
16. Repo-aware Kontextzugriff
17. Similarity Search über vergangene Fälle
18. Cross-Artefakt-Konsistenzprüfungen
19. UI für Review-Freigabe, Team-Notizen und Status

13. Roadmap, Delivery-Plan und Projektorganisation

13.1 Empfohlene Projektphasen

Phase	Dauer	Ziel	Ergebnisse
Phase 0 – Exploration	1 Woche	Review-Typen und	Beispielfälle, Zielbilder,

Phase	Dauer	Ziel	Ergebnisse
		Beispielmaterial sammeln	Output-Schema
Phase 1 – MVP Build	2–3 Wochen	brauchbaren Review-Core implementieren	MVP mit Findings, Tests, Fragen, Risiken
Phase 2 – Stabilisierung	1–2 Wochen	Outputs schärfen, Rule Packs nachziehen	präzisere Heuristiken, Review-Vergleiche
Phase 3 – v1 Ausbau	2–4 Wochen	Domänenwissen, History, mehr Kontexte	alltagstaugliches internes Tool
Phase 4 – v2/Premium	später	Integrationen, Repo-Kontext, Teamprozesse	skalierbares Quality-System

13.2 Konkreter Delivery-Plan für die ersten Wochen

- Woche 1: 15–25 reale Review-Beispiele sammeln, klassifizieren und Soll-Outputs definieren
- Woche 2: P0-Features aufsetzen, Rule-Packs grob skizzieren, erste Beispieltests fahren
- Woche 3: gegen echte oder alte Fälle kalibrieren, Schweregrade nachschärfen, Test-Gaps verbessern
- Woche 4: Domain Packs anreichern, UX/Export glätten, v1-Backlog priorisieren

13.3 Rollen im Mini-Projekt

- Product Owner / fachlicher Owner: priorisiert Review-Nutzen und Output-Struktur
- Technical Owner: verantwortet Rule Packs, Architektur und Datenmodell
- Review Curator: pflegt Beispiel-Fälle, Soll-Antworten und Fehlklassifikationen
- Optional UI/Tooling Support: baut Workbench, Export und History

14. Qualitätskriterien, Risiken und Anti-Goals

14.1 Qualitätskriterien

- Findings müssen priorisiert und umsetzbar sein
- der Assistant darf Unsicherheit nicht verschleiern
- Rückfragen müssen präzise und implementierungsrelevant sein
- Test-Gaps müssen realistisch und nicht generisch sein
- Ausgaben sollen zwischen Review-Typen konsistent bleiben
- keine erfundenen SAP-Objekte oder scheinbare Systemwahrheit

14.2 Hauptrisiken

- zu generische Findings ohne SAP-Tiefe
- zu viel LLM-Freiheit ohne Rule Pack-Geländer
- fehlender Kontext führt zu überzogenen oder irrelevanten Kritikpunkten
- Überfrachtung des Produkts mit zu vielen Modi vor MVP-Reife
- UI- oder Integrationsarbeit verdrängt den Kernnutzen der Findings-Qualität

14.3 Anti-Goals

- kein autonomer Refactoring-Bot
- kein Ersatz für erfahrenes Entwicklerurteil

- keine Scheinsicherheit bei Performance, Security oder Architektur
- kein generischer Chat ohne feste Review-Struktur
- kein Versuch, alle SAP-Technologien gleichzeitig gleich tief abzudecken

15. Beispiel-Fälle und erwartete Ergebnisse

15.1 Beispiel: ABAP-Snippet

Input: kleine Methode mit verschachtelter Logik, unscharfer Fehlerbehandlung und fehlenden Tests. Erwartete Ausgabe: priorisierte Findings zu Lesbarkeit, Ausnahmebehandlung, Seiteneffekten, Testbarkeit sowie konkreter Testlückenliste.

15.2 Beispiel: Fiori/UI5 Controller

Input: Controller mit überladener Event-Logik, direkter Modellmanipulation und unklaren Bindings. Erwartete Ausgabe: Findings zu Verantwortlichkeit, Wartbarkeit, UI-State-Handling, Testbarkeit und optionalem Refactoring-Pfad.

15.3 Beispiel: RAP/Fiori-Change

Input: Change-Plan für neue Action plus UI-Anpassung. Erwartete Ausgabe: Hinweise zu Behavior-Konsistenz, Services, Annotationen, Seiteneffekten, Testfällen und offenen Fachfragen.

15.4 Beispiel: Pre-Merge Review

Input: kleines gemischtes Änderungsset über CDS, Service und UI5. Erwartete Ausgabe: Top Findings, fehlende Tests, Go-with-caveats oder No-Go für den Merge im aktuellen Zustand.

16. Erfolgsmessung und nächste Ausbauschritte

16.1 Sinnvolle Erfolgskriterien

- höhere Review-Geschwindigkeit bei kleinen bis mittleren Änderungen
- weniger vergessene Tests, Rollenfragen oder Cross-Layer-Risiken
- höhere Konsistenz zwischen verschiedenen Reviews
- spürbar besser strukturierte technische Rückmeldungen
- mehr wiederverwendbare Review-Muster und weniger Ad-hoc-Denkarbeit

16.2 Nächste Ausbauschritte nach MVP

- Rule Packs gegen echte Fälle verfeinern
- Domain Packs aus deinen Logistik-/Service-Kontexten ergänzen
- History und Lernschleife aktivieren
- Review Assistant mit Field Change Accelerator und Skeleton Generator verzahnen
- erst danach über Repo-Integrationen, PR-Workflows oder Premium-Gates nachdenken

Der größte Hebel liegt nicht darin, den Assistant sofort maximal intelligent zu machen. Der größte Hebel liegt darin, eine stabile, wiederverwendbare Review-Sprache für ABAP/UI5-Fullstack-Arbeit aufzubauen. Wenn Findings, Test-Gaps, Risiken und nächste Schritte konsistent werden, entsteht aus einem einfachen Helfer sehr schnell ein echter Qualitätshebel.